RELATÓRIO DE AUDITORIA

Análise técnica do projeto Medical Review Sentiment Analysis — MLOps

Revisão do código, documentação, pipeline de ML, API, infraestrutura, testes, observabilidade e segurança.

Data da geração: 04/11/2026 14:38 E. South America Standard Time

Escopo: repositório de trabalho aberto em VS Code. A avaliação é documental e estática; não equivale a estudo clínico, benchmark controlado ou certificação de produção.

Conclusão executiva

O projeto reúne uma base funcional de engenharia — API FastAPI, treinamento PyTorch/Transformers, utilitários ONNX, tracking MLflow e stack de métricas Prometheus/Grafana — porém a documentação anterior apresentava como concluídos diversos componentes ausentes ou incompletos. O README foi reescrito para delimitar o escopo comprovável.

A conclusão científica central é negativa no sentido metodológico: não há evidência suficiente no repositório para afirmar acurácia, latência-alvo, validade clínica ou prontidão para produção. Existe código para treinar e calcular métricas, mas não há protocolo reproduzível de avaliação em conjunto de teste independente. A DAG Airflow não executa treinamento real. Sem checkpoint, o serviço ainda pode produzir inferências com cabeça classificadora recém-inicializada.

Classificação global: protótipo acadêmico em evolução; não apropriado para uso clínico ou decisões sobre pacientes.

1. Escopo e método

A revisão cruzou README, pyproject, Makefile, fonte Python, testes, DAG, Dockerfile, Compose, Prometheus/Grafana e inventário de arquivos. As afirmações foram classificadas como implementadas, parciais, ausentes ou não demonstradas. Não foram lidos dados médicos brutos nem arquivos .env; o arquivo de exemplo foi inspecionado apenas quanto à presença de credenciais e seu valor foi substituído por placeholders.

- Verificação estática do fluxo de dados, treinamento, inferência e contratos da API.
- Verificação de consistência de estrutura, scripts documentados, dependências e configurações declaradas.
- Diferenciação entre existência de código, execução comprovada e qualidade preditiva medida.
- A geração deste documento não valida um modelo, não substitui revisão humana e não constitui avaliação regulatória ou clínica.

Limite de evidência de execução

A tentativa de execução atual de pytest parou durante a coleta sem fornecer resultado final confiável nesta sessão. Um resultado anterior registrado no histórico foi 9 testes aprovados e 1 ignorado, mas ocorreu antes das últimas mudanças de observabilidade; não deve ser apresentado como verificação atual. Ruff e Mypy também não foram confirmados novamente após as mudanças. Docker/Compose e o dashboard após a última edição tampouco foram revalidados.

2. Inventário técnico

Área	Implementação observada	Estado / ressalva
Dados	Download Kaggle, fallback sintético; limpeza de CSV e split estratificado.	Parcial. Fallback de 10 linhas pode mascarar falha de aquisição; não é dataset científico.
Modelo	AutoModelForSequenceClassification com DistilBERT e 3 classes; treino com AdamW.	Código presente; qualidade e checkpoint não são garantidos.
Treinamento	CLI em src.models.train_pipeline; salva state_dict e loga algumas métricas.	Separação 80/20 treino-validação; sem avaliação independente e manifestos.
Inferência	Predictor PyTorch, saída de classe, confiança e scores.	Carrega pesos se disponíveis; sem checkpoint segue com cabeça inicializada.
Avaliação	Helpers sklearn para accuracy, precision, recall, F1 e confusion matrix.	Sem comando end-to-end que avalie um test set persistido.
ONNX	Exportação, quantização dinâmica e benchmark PyTorch/ONNX.	Utilitários independentes; API serve PyTorch, não ONNX.
Tracking	MLflow registra métricas/params em treinos e helper de metadata.	Sem registro/seleção de modelos no Registry comprovado.
API	FastAPI: health, predict, batch e metrics; Pydantic valida requests.	Health não é readiness; batch sequencial e sem limite de tamanho agregado.
Orquestração	DAG Airflow com preparação, tarefa nominal de treino e avaliação.	Treino só conta amostras; avaliação só registra retorno. Pipeline não treina.
Observabilidade	Métricas Prometheus de HTTP/inferência e dashboard provisionado.	Sinais operacionais, não métricas de qualidade, drift ou segurança clínica.
CI	GitHub Actions com Python 3.11, Ruff, Mypy, pytest e coverage.	Não é pipeline de deploy; resultado atual da execução não foi confirmado.

3. Arquitetura e fluxo de execução

O fluxo pretendido é CSV → limpeza/rotulagem → tokenização Hugging Face → fine-tuning PyTorch → checkpoint local → FastAPI. O checkpoint é state_dict; tokenizer/modelo base são carregados pelo identificador Hugging Face. O serviço expõe predição e métricas técnicas. Compose agrega MLflow/PostgreSQL, Airflow, Prometheus e Grafana.

Detalhes relevantes

- O pipeline de treino tem defaults próprios: duas épocas, batch size 8, max_length 128, divisão estratificada 80/20 e random_state 42; isso difere de defaults descritos em partes antigas da documentação.
- O módulo de preprocessing oferece divisão treino/validação/teste, mas o CLI de treinamento não utiliza esse método; ele retém apenas treino e validação.
- O treinamento chama mlflow.log_metrics sem encapsular claramente o ciclo inteiro em start_run no método mostrado; comportamento depende da configuração/ambiente de tracking.
- O endpoint batch itera predição individual; a inferência síncrona é chamada em handlers async, podendo bloquear o event loop.
- A resposta de erro de predição inclui str(exception) no detalhe HTTP; isso pode expor caminhos/detalhes internos.
- model_version existe no schema/exemplo, mas não direciona carregamento nem versionamento do modelo.
- O Compose usa defaults de desenvolvimento, imagens latest e senha estática; requer hardening antes de exposição.

4. Avaliação científica do modelo

A presença de uma arquitetura Transformer não é evidência de eficácia. Sem definição clara do conjunto de dados, origem/licença, idioma, classes, tamanho após limpeza, prevalência de rótulos, split independente, métricas calculadas e comparação com baseline, não é possível estimar generalização. O README antigo afirmava acurácia de 92–96%, latência específica e tamanho de BERT-base; essas afirmações não são sustentadas pelo código inspecionado. O modelo configurado é DistilBERT e não existe relatório versionado que comprove aquelas métricas.

Riscos de validade

- Fallback sintético de dez registros não deve entrar silenciosamente no fluxo de treino nem ser confundido com observações reais.
- Split estratificado pode falhar para classes com poucos exemplos; deduplicação apenas por texto não impede duplicatas semânticas ou vazamento entre partições.
- Não há validação de domínio, idioma, anotação, acordo entre anotadores, viés demográfico ou calibração de confiança.
- Não há evidência de teste clínico, revisão por especialistas, análise de danos ou governança de dados sensíveis.

Protocolo mínimo recomendado

(1) congelar e documentar dataset/licença/proveniência; (2) definir política de exclusão de duplicatas e um split independente por paciente/fonte quando aplicável; (3) comparar baseline majority-class e TF-IDF; (4) reportar matriz de confusão, macro-F1, métricas por classe e intervalos de confiança; (5) medir calibração e analisar erros por idioma/subgrupo; (6) registrar hash do dataset, commit, seed, tokenizer, dependências, configuração e artefato; (7) obter revisão humana especializada antes de qualquer uso em contexto de saúde.

5. README: validação das afirmações

Afirmação antiga	Veredito	Correção aplicada
BERT-base, 410 MB e 92–96% de accuracy	Incorreta/não demonstrada	Nome real DistilBERT; métricas removidas como alegação factual.
Prefect, Streamlit, TorchServe, DuckDB, GE, Evidently	Ausente	Listados explicitamente como não implementados.
Airflow executa treino e avaliação	Enganosa	DAG descrita como esqueleto; tarefa não treina nem avalia.
MLflow Model Registry	Não comprovado	README fala apenas em tracking; sem registry/serving.
make train/evaluate	Invocação incompatível	Documentado CLI real train_pipeline e lacuna de avaliação.
Python 3.11+	Incompatível	Requisito corrigido para >=3.11,<3.14.
Documentação/árvore listava módulos inexistentes	Desatualizada	Árvore real resumida no README e docs/ARCHITECTURE.md atualizada.
Credenciais Kaggle no .env.example	Incidente de segurança potencial	Substituído por placeholders; rotação continua necessária se valor era válido/exposto.

Outras correções: setup/execução, endpoints, limites do model_version, comportamento quando checkpoint ausente, Compose e diferenças entre monitoramento de serviço e avaliação de modelo. A documentação não afirma que imagens/serviços foram implantados após as alterações mais recentes.

6. Segurança, privacidade e confiabilidade

Risco	Evidência / impacto	Ação recomendada
Segredo em configuração versionada	Valor com formato de chave Kaggle apareceu em .env.example; valor não reproduzido.	Revogar/rotacionar já; inspecionar histórico Git e artefatos publicados; usar secret manager.
Checkpoint opcional	Sem pesos, classificador recém-inicializado pode emitir confiança enganosa.	Falhar fechado; readiness verifica checkpoint, hash, compatibilidade e versão aprovada.
Dados sensíveis	Reviews de saúde podem conter identificadores e informação sensível.	Minimização, controle de acesso, retenção curta, redaction de logs e avaliação legal/ética.
Dependências/imagem	Imagem runtime instala dependências pesadas do projeto; airflow no mesmo ambiente da API.	Separar imagens por serviço, lock reproducível, SBOM, scan e versões fixas.
Segredos de Compose	Senha Postgres/Grafana de desenvolvimento estática e portas publicadas.	Segredos externos, rede restrita, autenticação forte e sem exposição pública por padrão.
Serialização	torch.load usa arquivo local de pesos.	Aceitar apenas artefatos confiáveis e controlar integridade/proveniência.
Erros HTTP	Exceção é incluída no detalhe da resposta.	Retornar mensagem genérica e guardar diagnóstico sanitizado em log protegido.

7. Testes e evidências

A suíte disponível é pequena: API health/validação, API com predictor fake, limpeza de dados e um teste de modelo ignorado por depender de pesos. Isso é útil para contratos básicos, mas não testa qualidade do modelo, integração ONNX, pipeline Airflow funcional, registro MLflow ou robustez de produção. O relatório coverage.xml observado anteriormente indicava cobertura abaixo de 50%, mas era artefato pré-existente e não foi refeito nesta auditoria.

Estado desta revisão

- Confirmado por inspeção: arquivos e implementações descritas neste relatório, incluindo os limites da DAG e predictor.
- Tentativa recente: pytest iniciou coleta, sem resultado conclusivo retornado; não declarar pass/fail atual.

- Histórico da sessão: execução anterior reportada como 9 passed, 1 skipped antes das últimas alterações; evidência histórica, não revalidação.
- Não confirmado após mudanças: Ruff/Mypy, build Docker, validação Compose e funcionamento do dashboard provisionado.
- Artefatos locais de dados/pesos existem na máquina, mas estão ignorados pelo Git; não se presume distribuição em clone limpo.

8. Recomendações priorizadas

Prioridade	Ação	Critério de conclusão
P0 — Segurança	Revogar chave potencialmente exposta; confirmar limpeza do histórico e imagens.	Credencial revogada, secret scanning e credenciais só via secret manager.
P0 — Integridade de inferência	Bloquear serviço/predição sem checkpoint aprovado; readiness separado de liveness.	Teste garante falha clara sem modelo e expõe hash/versão sem segredo.
P1 — Validade científica	Construir dataset/split de teste documentados, baseline e relatório por classe.	Resultados reproduzíveis associados a commit/dados/configuração e revisão humana.
P1 — Pipeline	Fazer DAG executar treino e avaliação reais, corrigir executor e banco metadata.	Execução de ponta a ponta idempotente, artefatos e falhas observáveis.
P1 — Serviço	Limites de batch, inferência fora do event loop, timeout e erros sanitizados.	Testes de carga/limites e estabilidade sob concorrência.
P2 — Reprodutibilidade	Lock de dependências, versões de imagens, manifesto de dados/modelo e seeds.	Clone limpo reproduz setup; versões e origem de artefatos auditáveis.
P2 — Operação	Segredos externos, auth/TLS, restrição de portas e política de logs/retention.	Checklist de segurança aprovado antes de ambiente acessível externamente.

9. Estado da documentação entregue

README.md foi refeito em português, agora apresenta os comandos reais e deixa explícitas as capacidades ausentes, o comportamento de fallback, a ausência de checkpoint, as limitações de avaliação e os riscos de deployment. docs/ARCHITECTURE.md foi alinhado à implementação, removendo diagramas aspiracionais que descreviam componentes inexistentes. .env.example teve o valor de credencial substituído por placeholders; isso não revoga uma chave que possa ter sido verdadeira.

Arquivos principais de evidência

src/models/train_pipeline.py; src/models/train_bert.py; src/models/predict.py; src/models/evaluate.py; src/models/onnx_converter.py; src/models/benchmark.py; src/data/dataset.py; src/data/preprocessing.py; src/api/app.py; src/api/routes.py; src/api/schemas.py; dags/sentiment_training_dag.py; docker-compose.yml; Dockerfile; pyproject.toml; Makefile; .github/workflows/tests.yml; tests/.

Parecer final

O valor do projeto está na integração didática de componentes de ML/MLOps e na oportunidade concreta de evoluí-lo. Seu estado atual deve ser descrito como protótipo: primeiro fortalecer integridade de modelo e credenciais; depois estabelecer avaliação científica reproduzível; por fim operacionalizar DAG, serving e segurança. Até lá, números de desempenho e qualquer interpretação clínica devem permanecer fora de afirmações públicas.